

```

/*DELIMITER $$

CREATE FUNCTION clean_text(input_text TEXT)

RETURNS TEXT

DETERMINISTIC

BEGIN

    DECLARE cleaned TEXT;

    SET cleaned = TRIM(input_text);

    SET cleaned = REPLACE(REPLACE(REPLACE(cleaned, '\n', ''), '\r', ''), '\t', '');

    SET cleaned = REGEXP_REPLACE(cleaned, '[^a-zA-Z0-9 ]', '');

    SET cleaned = LOWER(cleaned);

    RETURN cleaned;

END$$

DELIMITER ;

```

```

CREATE INDEX idx_events_session ON events(session_id);

CREATE INDEX idx_orders_userid ON orders(user_id);

CREATE INDEX idx_orderitems_orderid ON order_items(order_id);

CREATE INDEX idx_products_id ON products(id);

CREATE INDEX idx_events_event_type ON events(event_type);

CREATE INDEX idx_events_uri ON events(uri);

CREATE INDEX idx_events_traffic_source ON events(traffic_source);

CREATE INDEX idx_orders_created_at ON orders(created_at);

*/

-- CMO ANALYSIS -----
-- Acquisition Analysis -----

create view Acquisition_Analysis as

```

```

SELECT
    traffic_source,
    COUNT(DISTINCT session_id) AS total_sessions,
    COUNT(CASE
        WHEN event_type = 'purchase' THEN event_type
        ELSE NULL
    END) total_orders,
    COUNT(DISTINCT CASE
        WHEN user_id IS NOT NULL THEN session_id
        END) AS logged_in_customers,
    COUNT(DISTINCT CASE
        WHEN user_id IS NULL THEN session_id
        END) AS guest_sessions
FROM
    events
GROUP BY traffic_source;

```

-- ----- Channel Quality -----

-- CVR traffic source

create view traffic_source as

```

SELECT
    e.traffic_source,
    COUNT(DISTINCT session_id) AS total_sessions,
    COUNT(DISTINCT CASE
        WHEN event_type = 'purchase' THEN session_id
        END) AS total_order_sessions,

```

```
round(CAST(COUNT(DISTINCT CASE
    WHEN event_type = 'purchase' THEN session_id
END)
AS DECIMAL (18 , 4 )) / COUNT(DISTINCT session_id), 4) AS CVR
FROM
    events AS e
GROUP BY e.traffic_source
ORDER BY total_sessions DESC;
```

-- CVR browser

create view CVR_browser as

```
SELECT
    e.browser,
    COUNT(DISTINCT session_id) AS total_sessions,
    COUNT(DISTINCT CASE
        WHEN event_type = 'purchase' THEN session_id
    END) AS total_order_sessions,
    CAST(COUNT(DISTINCT CASE
        WHEN event_type = 'purchase' THEN session_id
    END)
    AS DECIMAL (18 , 4 )) / COUNT(DISTINCT session_id) AS CVR
FROM
    events AS e
GROUP BY e.browser
ORDER BY total_sessions DESC;
```

-- ----- B.Customer Value Analysis -----

-- CLV

create view CLV as

WITH OrderProfit AS (

SELECT

o.user_id,

oi.order_id,

SUM(oi.sale_price) AS total_revenue_per_order,

SUM(p.cost) AS total_cost_per_order,

(SUM(oi.sale_price) - SUM(p.cost)) AS total_profit_per_order

FROM

order_items oi

JOIN

products p ON oi.product_id = p.id

JOIN

orders o ON oi.order_id = o.order_id

WHERE

o.user_id IS NOT NULL

AND o.status in('Complete', 'Shipped')

GROUP BY

o.user_id, oi.order_id

)

SELECT

user_id,

COUNT(order_id) AS total_orders,

```
round(SUM(total_revenue_per_order),2) AS total_lifetime_revenue,
round(SUM(total_cost_per_order),2) AS total_lifetime_cost,
round(SUM(total_profit_per_order),2) AS historical_clv
FROM
    OrderProfit
GROUP BY
    user_id
ORDER BY
    user_id;
```

-- Channels CLV

create or replace view Channels_CLV as

```
WITH CustomerCLV AS (
    SELECT
        o.user_id,
        (SUM(oi.sale_price) - SUM(p.cost)) AS historical_clv
    FROM
        order_items oi
    JOIN
        products p ON oi.product_id = p.id
    JOIN
        orders o ON oi.order_id = o.order_id
    WHERE
        o.user_id IS NOT NULL
        AND o.status in('Complete', 'Shipped','Purchased')
    GROUP BY
```

```

        o.user_id
    ),
    UserFirstTouch AS (
        SELECT
            user_id,
            traffic_source
        FROM (
            SELECT
                user_id,
                traffic_source,
                ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY sequence_number ASC) as
event_rank
            FROM
                events
            WHERE
                user_id IS NOT NULL
                AND traffic_source IS NOT NULL
        ) AS RankedEvents
        WHERE
            event_rank = 1
    )

```

```

SELECT
    ft.traffic_source,
    round(COUNT(DISTINCT clv.user_id),2) AS total_acquired_customers,
    round(AVG(clv.historical_clv),2) AS average_clv_per_customer,

```

```
        round(SUM(clv.historical_clv),2) AS total_clv_from_channel
FROM
    CustomerCLV clv
JOIN
    UserFirstTouch ft ON clv.user_id = ft.user_id
GROUP BY
    ft.traffic_source
ORDER BY
    average_clv_per_customer DESC;
```

-- New vs. Returning

create or replace view New_vs_Returning as

WITH OrderRevenue AS (

```
    SELECT
        o.order_id,
        o.user_id,
        o.created_at,
        SUM(oi.sale_price) AS order_revenue
```

FROM

```
    orders o
```

JOIN

```
    order_items oi ON o.order_id = oi.order_id
```

WHERE

```
    oi.status in('Complete', 'Shipped','Processing')
```

GROUP BY

```
    o.order_id, o.user_id, o.created_at
```

),

TaggedOrders AS (

SELECT

order_id,

user_id,

order_revenue,

ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY created_at ASC) AS order_rank

FROM

OrderRevenue

),

FinalMetrics AS (

SELECT

CASE

WHEN order_rank = 1 THEN 'New Customer'

ELSE 'Returning Customer'

END AS customer_type,

COUNT(order_id) AS total_orders,

SUM(order_revenue) AS total_revenue

FROM

TaggedOrders

GROUP BY

customer_type

)

SELECT

customer_type,


```

total_orders,
round(total_revenue,2) total_revenue,
round((total_revenue / total_orders),2) AS AOV,
round((CAST(total_revenue AS DECIMAL(18,2)) / SUM(total_revenue) OVER ()) * 100,2) AS
revenue_percentage
FROM
    FinalMetrics
ORDER BY
    customer_type;

```

-- Cohort Analysis

create view Cohort_Analysis as

WITH ValidPurchases AS (

SELECT DISTINCT

o.user_id,

DATE_FORMAT(o.created_at, '%Y-%m-01') AS purchase_month

FROM

orders o

JOIN

order_items oi ON o.order_id = oi.order_id

WHERE

o.status <> 'returned'

AND o.user_id IS NOT NULL

),

UserCohorts AS (

SELECT

```

        user_id,
        MIN(purchase_month) AS cohort_month
    FROM
        ValidPurchases
    GROUP BY
        user_id
),
RetentionData AS (
    SELECT
        c.user_id,
        c.cohort_month,
        p.purchase_month,
        TIMESTAMPDIFF(MONTH, c.cohort_month, p.purchase_month) AS month_index
    FROM
        UserCohorts c
    JOIN
        ValidPurchases p ON c.user_id = p.user_id
)
SELECT
    cohort_month,
    COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END) AS cohort_size,
    (CAST(COUNT(DISTINCT CASE WHEN month_index = 1 THEN user_id END) AS
    DECIMAL(10,4))
    /
    COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
    month_1_retention,

```

(CAST(COUNT(DISTINCT CASE WHEN month_index = 2 THEN user_id END) AS
DECIMAL(10,4))

/

COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_2_retention,

(CAST(COUNT(DISTINCT CASE WHEN month_index = 3 THEN user_id END) AS
DECIMAL(10,4))

/

COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_3_retention,

(CAST(COUNT(DISTINCT CASE WHEN month_index = 4 THEN user_id END) AS
DECIMAL(10,4))

/

COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_4_retention,

(CAST(COUNT(DISTINCT CASE WHEN month_index = 5 THEN user_id END) AS DECIMAL(10,4))

/

COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_5_retention,

(CAST(COUNT(DISTINCT CASE WHEN month_index = 6 THEN user_id END) AS DECIMAL(10,4))

/

COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_6_retention,

(CAST(COUNT(DISTINCT CASE WHEN month_index = 7 THEN user_id END) AS DECIMAL(10,4))

/

COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_7_retention,

(CAST(COUNT(DISTINCT CASE WHEN month_index = 8 THEN user_id END) AS
DECIMAL(10,4))

```

/
COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_8_retention,
(CAST(COUNT(DISTINCT CASE WHEN month_index = 9 THEN user_id END) AS DECIMAL(10,4))
/
COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_9_retention,
(CAST(COUNT(DISTINCT CASE WHEN month_index = 10 THEN user_id END) AS
DECIMAL(10,4))
/
COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_10_retention,
(CAST(COUNT(DISTINCT CASE WHEN month_index = 11 THEN user_id END) AS
DECIMAL(10,4))
/
COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_11_retention,
(CAST(COUNT(DISTINCT CASE WHEN month_index = 12 THEN user_id END) AS
DECIMAL(10,4))
/
COUNT(DISTINCT CASE WHEN month_index = 0 THEN user_id END)) * 100 AS
month_12_retention

```

FROM

RetentionData

GROUP BY

cohort_month

ORDER BY

```
cohort_month;

-- AOV over months
create view AOV_over_months as
SELECT
    DATE_FORMAT(o.created_at, '%Y-%m') AS purchase_month_year,
    COUNT(DISTINCT o.order_id) AS total_orders,
    AVG(oi.order_revenue) AS average_order_value
FROM
    orders o
    JOIN
    (SELECT
        order_id, SUM(sale_price) AS order_revenue
    FROM
        order_items
    GROUP BY order_id) oi ON o.order_id = oi.order_id
WHERE
    o.status <> 'returned'
GROUP BY purchase_month_year
ORDER BY purchase_month_year;

-- AOV over years
create view AOV_over_years as
SELECT
    DATE_FORMAT(o.created_at, '%Y') AS purchase_year,
    COUNT(DISTINCT o.order_id) AS total_orders,
    AVG(oi.order_revenue) AS average_order_value
```

```

FROM
    orders o
    JOIN
    (SELECT
        order_id, SUM(sale_price) AS order_revenue
    FROM
        order_items
    GROUP BY order_id) oi ON o.order_id = oi.order_id
WHERE
    o.status <> 'returned'
GROUP BY purchase_year
ORDER BY purchase_year;

```

-- ----- C.Marketing Behavior -----

```

create view Marketing_Behavior as
WITH SessionFacts AS (
    SELECT
        session_id,
        MAX(traffic_source) AS traffic_source,
        MAX(CASE
            WHEN event_type = 'cart' THEN 1
            ELSE 0
        END) AS reached_cart,
        MAX(CASE
            WHEN event_type = 'purchase' THEN 1
            ELSE 0

```

```

        END) AS did_purchase
FROM
    events
GROUP BY
    session_id
)
SELECT
    traffic_source,
        COUNT(session_id) AS total_sessions_reached_cart,
        SUM(did_purchase) AS total_sessions_purchased,
        (CAST(SUM(did_purchase) AS DECIMAL(18,4)) / COUNT(session_id)) * 100 AS
cart_conversion_rate_percent
FROM
    SessionFacts
WHERE
    reached_cart = 1
    AND traffic_source IS NOT NULL
GROUP BY
    traffic_source
ORDER BY
    cart_conversion_rate_percent ASC;

```

-- Top region by Revenue & AOV

create view Top_region_by_Revenue_AOV as

WITH OrderRevenue AS (

```

    SELECT

```

```

        o.order_id,
        o.user_id,
        SUM(oi.sale_price) AS order_revenue
FROM
    orders o
JOIN
    order_items oi ON o.order_id = oi.order_id
WHERE
    o.status <> 'returned'
GROUP BY
    o.order_id, o.user_id
),
UserOrders AS (
    SELECT
        u.country,
        u.state,
        u.city,
        o.order_id,
        o.order_revenue
    FROM
        OrderRevenue o
    JOIN
        users u ON o.user_id = u.id
    WHERE
        u.state IS NOT NULL AND o.user_id IS NOT NULL
)

```



```

SELECT
    country,
    state,
    city,
    COUNT(DISTINCT order_id) AS total_orders,
    round(SUM(order_revenue),2) AS total_revenue,
    round((SUM(order_revenue) / COUNT(DISTINCT order_id)),2) AS AOV
FROM
    UserOrders
GROUP BY
    state, country, city
ORDER BY
    total_revenue DESC
LIMIT 10;

```

```

-- CX ANALYSIS -----
-- ----- A.On-Site Friction Analysis ---
-- Funnel Drop-off -----

```

```

-- identifying the right shcema
create view identifying_the_right_shcema as
WITH SessionFacts AS (

```

```

    SELECT
        session_id,
        MAX(CASE WHEN event_type = 'home' THEN 1 ELSE 0 END) AS visited_home,
        MAX(CASE WHEN event_type = 'product' THEN 1 ELSE 0 END) AS viewed_product,

```

```
        MAX(CASE WHEN event_type = 'cart' THEN 1 ELSE 0 END) AS added_to_cart,  
        MAX(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END) AS did_purchase  
FROM  
    events  
GROUP BY  
    session_id  
)
```

```
SELECT  
    visited_home,  
    viewed_product,  
    added_to_cart,  
    did_purchase,  
    COUNT(*) AS session_count  
FROM  
    SessionFacts  
GROUP BY  
    visited_home,  
    viewed_product,  
    added_to_cart,  
    did_purchase  
ORDER BY  
    session_count DESC;  
-- funnel  
create view funnel as  
WITH SessionFacts AS (
```

```

SELECT
    session_id,
    MAX(CASE WHEN event_type = 'product' THEN 1 ELSE 0 END) AS viewed_product,
    MAX(CASE WHEN event_type = 'cart' THEN 1 ELSE 0 END) AS added_to_cart,
    MAX(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END) AS did_purchase
FROM
    events
GROUP BY
    session_id
),
FunnelCounts AS (
    SELECT
        SUM(viewed_product) AS step1_viewed_product,
        SUM(CASE
            WHEN viewed_product = 1 AND added_to_cart = 1 THEN 1
            ELSE 0
        END) AS step2_added_to_cart,
        SUM(CASE
            WHEN viewed_product = 1 AND added_to_cart = 1 AND did_purchase = 1 THEN 1
            ELSE 0
        END) AS step3_purchased
    FROM
        SessionFacts
)

SELECT

```

```

    step1_viewed_product,
    step2_added_to_cart,
    -- round((CAST(step2_added_to_cart AS DECIMAL(18,4)) / step1_viewed_product) * 100, 2)
AS product_to_cart_rate,
    step3_purchased
    -- round((CAST(step3_purchased AS DECIMAL(18,4)) / step2_added_to_cart) * 100, 2) AS
cart_to_purchase_rate
FROM
    FunnelCounts;

-- funnel trend
create view funnel_trend as
WITH SessionFacts AS (
    SELECT
        DATE_FORMAT(created_at, '%Y') over_years,
        session_id,
        MAX(CASE WHEN event_type = 'product' THEN 1 ELSE 0 END) AS viewed_product,
        MAX(CASE WHEN event_type = 'cart' THEN 1 ELSE 0 END) AS added_to_cart,
        MAX(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END) AS did_purchase
    FROM
        events
    GROUP BY
        session_id, DATE_FORMAT(created_at, '%Y')
),
FunnelCounts AS (
    SELECT
        over_years,

```

```

SUM(viewed_product) AS step1_viewed_product,
    SUM(CASE
        WHEN viewed_product = 1 AND added_to_cart = 1 THEN 1
        ELSE 0
    END) AS step2_added_to_cart,
    SUM(CASE
        WHEN viewed_product = 1 AND added_to_cart = 1 AND did_purchase = 1 THEN 1
        ELSE 0
    END) AS step3_purchased
FROM
    SessionFacts
    GROUP BY over_years
    ORDER BY over_years
)

SELECT
    over_years,
    step1_viewed_product,
    step2_added_to_cart,
    round((CAST(step2_added_to_cart AS DECIMAL(18,4)) / step1_viewed_product) * 100, 2) AS
product_to_cart_rate,
    step3_purchased,
    round((CAST(step3_purchased AS DECIMAL(18,4)) / step2_added_to_cart) * 100, 2) AS
cart_to_purchase_rate
FROM
    FunnelCounts;

```

```
-- Exit Rate

create view Exit_Rate as

WITH RankedEvents AS (

    SELECT

        session_id,

        uri,

        ROW_NUMBER() OVER(PARTITION BY session_id ORDER BY sequence_number DESC) AS
event_rank

    FROM

        events

),

ExitCounts AS (

    SELECT

        uri,

        COUNT(*) AS total_exits

    FROM

        RankedEvents

    WHERE

        event_rank = 1

        AND uri IS NOT NULL

    GROUP BY

        uri

),

PageViews AS (

    SELECT

        uri,
```

```

        COUNT(DISTINCT session_id) AS total_views
    FROM
        events
    WHERE
        uri IS NOT NULL
    GROUP BY
        uri
)
SELECT
    pv.uri,
    pv.total_views,
    COALESCE(ex.total_exits, 0) AS total_exits,
    (CAST(COALESCE(ex.total_exits, 0) AS DECIMAL(18,4)) / pv.total_views) * 100 AS
    exit_rate_percent
FROM
    PageViews pv
LEFT JOIN
    ExitCounts ex ON pv.uri = ex.uri
WHERE
    pv.total_views > 100
    and
    COALESCE(ex.total_exits, 0) <> 0
ORDER BY
    exit_rate_percent DESC;

```

-- ----- B.Dissatisfaction Analysis -----

```
-- states return rates

create view states_return_rates as

WITH StateOrders AS (

    SELECT

        u.state,

        o.order_id,

        o.returned_at

    FROM

        orders o

    JOIN

        users u ON o.user_id = u.id

    WHERE

        u.state IS NOT NULL

        AND year(o.shipped_at) <> '0000'

)

SELECT

    state,

    COUNT(order_id) AS total_shipped_orders,

    SUM(CASE

        WHEN year(returned_at) <> '0000' THEN 1

        ELSE 0

    END) AS total_returned_orders,

    (CAST(SUM(CASE

        WHEN year(returned_at) <> '0000' THEN 1

        ELSE 0

    END) AS DECIMAL(18,4))
```



```
    /  
    COUNT(order_id)) * 100 AS return_rate_percent  
FROM  
    StateOrders  
GROUP BY  
    state  
HAVING  
    COUNT(order_id) > 50  
ORDER BY  
    return_rate_percent DESC;
```

-- Order Return Rate - ORR

create view orr as

```
WITH OrderReturnStatus AS (  
    SELECT  
        order_id,  
        MAX(CASE  
            WHEN status = 'Returned' THEN 1  
            ELSE 0  
        END) AS has_a_return  
FROM  
    order_items  
GROUP BY  
    order_id  
)  
SELECT
```

```

        DATE_FORMAT(o.created_at, '%Y') over_years,
COUNT(o.order_id) AS total_shipped_orders,
        SUM(ors.has_a_return) AS orders_with_a_return,
        (CAST(SUM(ors.has_a_return) AS DECIMAL(18,4)) / COUNT(o.order_id)) * 100 AS
order_return_rate_percent
FROM
    orders o
JOIN
    OrderReturnStatus ors ON o.order_id = ors.order_id
WHERE
    year(o.shipped_at) <> '0000'
GROUP BY DATE_FORMAT(o.created_at, '%Y')
ORDER BY over_years;

```

-- Item Return Rate - IRR

create view irr as

```

SELECT
    DATE_FORMAT(o.created_at, '%Y') over_years,
COUNT(oi.id) AS total_shipped_items,
        SUM(CASE
            WHEN oi.status = 'Returned'
            THEN 1
            ELSE 0
        END) AS total_returned_items,
        (CAST(SUM(CASE WHEN oi.status = 'Returned' THEN 1 ELSE 0 END) AS DECIMAL(18,4)) /
COUNT(oi.id)) * 100 AS item_return_rate_percent
FROM

```

```
    order_items oi
JOIN
    orders o ON oi.order_id = o.order_id
WHERE
    year(o.shipped_at) <> '0000'
GROUP BY DATE_FORMAT(o.created_at, '%Y')
ORDER BY over_years;
```

-- Toxic Products

```
create view Toxic_Products as
WITH ItemProductStatus AS (
    SELECT
        p.name,
        oi.id AS item_id,
        (CASE
            WHEN oi.status = 'Returned' THEN 1
            ELSE 0
        END) AS is_returned
    FROM
        order_items oi
JOIN
        orders o ON oi.order_id = o.order_id
JOIN
        products p ON oi.product_id = p.id
WHERE
```

```

        year(o.shipped_at) <> '0000'
    )
SELECT
    name,
    COUNT(item_id) AS total_shipped_items,
    SUM(is_returned) AS total_returned_items,
    (CAST(SUM(is_returned) AS DECIMAL(18,4)) / COUNT(item_id)) * 100 AS return_rate_percent
FROM
    ItemProductStatus
GROUP BY
    name
HAVING
    return_rate_percent = 100.00000000
ORDER BY
    return_rate_percent DESC;
-- Return Rate by Category
create view Return_Rate_by_Category as
WITH ItemCategoryStatus AS (
    SELECT
        p.category,
        oi.id AS item_id,
        (CASE
            WHEN oi.status = 'Returned' THEN 1
            ELSE 0
        END) AS is_returned
    FROM

```

```

        order_items oi
    JOIN
        orders o ON oi.order_id = o.order_id
    JOIN
        products p ON oi.product_id = p.id
    WHERE
        year(o.shipped_at) <> '0000'
)
SELECT
    category,
        COUNT(item_id) AS total_shipped_items,
        SUM(is_returned) AS total_returned_items,
        (CAST(SUM(is_returned) AS DECIMAL(18,4)) / COUNT(item_id)) * 100 AS
return_rate_percent
FROM
    ItemCategoryStatus
GROUP BY
    category
ORDER BY
    total_returned_items DESC;

```

-- Return Rate by Geography

create view Return_Rate_by_Geography as

WITH ShippedItemStatus AS (

SELECT

oi.id AS item_id,

```

        oi.product_id,
            (CASE
                WHEN oi.status = 'Returned' THEN 1
                ELSE 0
            END) AS is_returned
FROM
    order_items oi
JOIN
    orders o ON oi.order_id = o.order_id
WHERE
    year(o.shipped_at) <> '0000'
),

```

```

ItemDistribution AS (
    SELECT
        sis.item_id,
        sis.is_returned,
        p.distribution_center_id
    FROM
        ShippedItemStatus sis
    JOIN
        products p ON sis.product_id = p.id
)
SELECT
    dc.name AS distribution_center_name,
    COUNT(id.item_id) AS total_shipped_items,

```

```
        SUM(id.is_returned) AS total_returned_items,
        (CAST(SUM(id.is_returned) AS DECIMAL(18,4)) / COUNT(id.item_id)) * 100 AS
return_rate_percent
FROM
    ItemDistribution id
JOIN
    distribution_centers dc ON id.distribution_center_id = dc.id
GROUP BY
    dc.name
ORDER BY
    return_rate_percent DESC;
```

create or replace view calendar as

```
select distinct date_format(created_at,'%Y-%m-%d') calendar
from events
group by calendar
order by calendar;
```

CREATE VIEW fact_order_details AS

SELECT

```
o.order_id,
o.user_id,
oi.product_id,
oi.created_at,
oi.shipped_at,
```

```
oi.returned_at,  
oi.sale_price,  
p.cost,  
o.status  
FROM  
orders o  
JOIN  
order_items oi ON o.order_id = oi.order_id  
JOIN  
products p ON p.id = oi.product_id;
```

```
CREATE or replace VIEW vw_CleanedEvents AS
```

```
WITH StitchedSessions AS (  
SELECT  
*,  
MAX(user_id) OVER (PARTITION BY session_id) AS stitched_user_id  
FROM  
events  
)
```

```
SELECT  
id,  
session_id,
```



```

created_at,
sequence_number,
browser,
ip_address,
traffic_source,
CAST(
    CASE
        WHEN event_type = 'product' THEN
            REGEXP_REPLACE(uri, '^[0-9]', '')
        ELSE NULL
    END
    AS UNSIGNED) AS product_id,
stitched_user_id,
CASE
    WHEN stitched_user_id IS NULL THEN 'Guest'
    ELSE 'Known'
END AS user_type,
    event_type AS event_type,
    uri AS uri

FROM

    StitchedSessions;

CREATE VIEW vw_Dim_Users_with_First_Touch AS
WITH UserFirstTouch AS (
    SELECT

```

```
    user_id,
    traffic_source AS first_touch_source
FROM (
    SELECT
        user_id,
        traffic_source,
        ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY sequence_number ASC) as
event_rank
    FROM
        events
    WHERE
        user_id IS NOT NULL
        AND traffic_source IS NOT NULL
) AS RankedEvents
WHERE
    event_rank = 1
)
SELECT
    u.id AS user_id,
    u.age,
    u.state,
    u.city,
    u.country,
    COALESCE(ft.first_touch_source, 'Unknown') AS acquiring_channel
FROM
    users u
```

LEFT JOIN

UserFirstTouch ft ON u.id = ft.user_id;

CREATE VIEW vw_CohortAnalysis_Long AS

WITH ValidPurchases AS (

SELECT DISTINCT

o.user_id,

DATE_FORMAT(o.created_at, '%Y-%m-01') AS purchase_month

FROM

orders o

JOIN

order_items oi ON o.order_id = oi.order_id

WHERE

o.status <> 'returned' or 'cancelled'

AND o.user_id IS NOT NULL

),

UserCohorts AS (

SELECT

user_id,

MIN(purchase_month) AS cohort_month

FROM

ValidPurchases

GROUP BY

user_id

),

RetentionData AS (

SELECT

c.user_id,

c.cohort_month,

p.purchase_month,

TIMESTAMPDIFF(MONTH, c.cohort_month, p.purchase_month) AS month_index

FROM

UserCohorts c

JOIN

ValidPurchases p ON c.user_id = p.user_id

),

CohortCounts AS (

SELECT

cohort_month,

month_index,

COUNT(DISTINCT user_id) AS retained_users

FROM

RetentionData

GROUP BY

cohort_month, month_index

)

SELECT

cohort_month,

month_index,

retained_users,

FIRST_VALUE(retained_users) OVER (

```
    PARTITION BY cohort_month
    ORDER BY month_index ASC
) AS cohort_size,
    (CAST(retained_users AS DECIMAL(18,4)) /
    FIRST_VALUE(retained_users) OVER (
        PARTITION BY cohort_month
        ORDER BY month_index ASC
    )) * 100 AS retention_percentage
FROM
    CohortCounts
ORDER BY
    cohort_month, month_index;
```